



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

PROJECT 1 REPORT

ECTE441 – Intelligent Control

Student Name	Student Number
Yasan Gunathilaka	7549246

02nd May, 2025

I. Table of Contents

I.	TABLE OF CONTENTS	2
II.	TABLE OF FIGURES	3
1.	INTRODUCTION	4
2.	PROBLEM DEFINITION	4
2.1.	STAGE 1: CONTROL SYSTEM SIMULATION.....	4
2.2.	STAGE 2: FUZZY INFERENCE SYSTEM.....	4
2.3.	STAGE 3: FUZZY CONTROL OF AN ELECTRIC MOTOR.....	4
2.4.	STAGE 4: TSK FUZZY CONTROL OF AN ELECTRIC MOTOR	5
3.	PROCEDURES & METHODOLOGIES	5
3.1.	STAGE 1: CONTROL SYSTEM SIMULATION.....	5
3.2.	STAGE 2: FUZZY INFERENCE SYSTEM.....	6
3.3.	STAGE 3: FUZZY CONTROL OF AN ELECTRIC MOTOR.....	7
3.4.	STAGE 4: TSK FUZZY CONTROL OF AN ELECTRIC MOTOR	8
4.	RESULTS & DISCUSSION.....	9
4.1.	STAGE 1: CONTROL SYSTEM SIMULATION.....	9
4.2.	STAGE 2: FUZZY INFERENCE SYSTEM.....	12
4.3.	STAGE 3: FUZZY CONTROL OF AN ELECTRIC MOTOR.....	13
4.4.	STAGE 4: TSK FUZZY CONTROL OF AN ELECTRIC MOTOR	15
5.	CONCLUSION	16
6.	APPENDIX.....	17
A.	STAGE 1: CONTROL SYSTEM SIMULATION	17
B.	STAGE 2: FUZZY INFERENCE SYSTEM	17
C.	STAGE 3: FUZZY CONTROL OF AN ELECTRIC MOTOR	21
D.	STAGE 4: TSK FUZZY CONTROL OF AN ELECTRIC MOTOR.....	24

II. Table of Figures

FIGURE 1 - PI SYSTEM WITHOUT LOAD	5
FIGURE 2 - PI SYSTEM WITH LOAD.....	6
FIGURE 3 - FUZZY LOGIC CONTROL SYSTEM FOR MOTOR WITHOUT LOAD	7
FIGURE 4 - FUZZY LOGIC CONTROL SYSTEM FOR MOTOR WITH LOAD.....	8
FIGURE 5 - TSK LOGIC CONTROL SYSTEM FOR MOTOR WITH LOAD	9
FIGURE 6 - FINE-TUNED PI CONTROLLER WITHOUT LOAD	10
FIGURE 7 – FINE-TUNED PI CONTROLLER WITH LOAD	11
FIGURE 8 - FINE-TUNED FUZZY CONTROL SYSTEM WITHOUT LOAD	14
FIGURE 9 - FINE-TUNED FUZZY CONTROL SYSTEM WITH LOAD	14
FIGURE 10 – FINE-TUNED TSK CONTROL SYSTEM WITH LOAD.....	15
FIGURE 11 - RISE TIME AND MAXIMUM OVERSHOOT OF FINE-TUNED PI CONTROLLER (WITHOUT LOAD).....	17
FIGURE 12 - RISE TIME AND MAXIMUM OVERSHOOT OF FINE-TUNED PI CONTROLLER (WITH LOAD)	17
FIGURE 13 - DEFINING PROPERTIES OF MAMDANI FIS - TIPPER.....	17
FIGURE 14 - DEFINING MEMBERSHIP FUNCTION FOR SERVICE (INPUT 1).....	18
FIGURE 15 - DEFINING MEMBERSHIP FUNCTIONS FOR FOOD (INPUT 2)	18
FIGURE 16 - DEFINING MEMBERSHIP FUNCTIONS FOR TIP (OUTPUT)	18
FIGURE 17 – DEFINING RULE BASE FOR FIS	19
FIGURE 18 - OBSERVING RULE INFERENCE PROCESS FOR DIFFERENT INPUTS.....	19
FIGURE 19 – GENERATING CONTROL SURFACE FOR INPUTS VS OUTPUT OF THE MAMDANI FIS	19
FIGURE 20 – CONVERTING MAMDANI CONTROL SURFACE TO A TSK (SUGENO) FIS	20
FIGURE 21 - COMPARISON BETWEEN TIPPING SYSTEM IN PROJECT (TOP) AND WORKSHOP I (BOTTOM)	20
FIGURE 22 - DEFINING PROPERTIES OF MAMDANI FIS - MOTOR_CONTROL	21
FIGURE 23 - MEMBERSHIP FUNCTIONS FOR THE INPUTS.....	21
FIGURE 24 - MEMBERSHIP FUNCTIONS FOR THE OUTPUT.....	21
FIGURE 25 - RULE BASE SYSTEM FOR THE MOTOR CONTROL FIS	22
FIGURE 26 - OBSERVING RULE INFERENCE PROCESS FOR MOTOR CONTROL.....	23
FIGURE 27 - CONTROL SURFACE OF FUZZY LOGIC CONTROLLER FOR MOTOR CONTROL.....	23
FIGURE 28 - 5 PI CONTROLLERS FOR EACH SECTION	24
FIGURE 29 - DEFINING PROPERTIES OF TSK FIS - TSK_MOTOR_CONTROL.....	25
FIGURE 30 - MEMBERSHIP FUNCTIONS FOR THE INPUTS.....	25
FIGURE 31 - OUTPUT CONSTANTS (CRISP OUTPUT)	25
FIGURE 32 - RULE BASE SYSTEM FOR THE TSK MOTOR CONTROL FIS (CRISP OUTPUTS).....	26
FIGURE 33 - CONTROL SURFACE OF TSK CONTROLLER FOR MOTOR CONTROL.....	26

1. Introduction

The project helps the student put the theories they had learnt in the lectures and workshops of ECTE441 to practical use, aiming to be a hands-on learning experience using MATLAB and Simulink to simulate, evaluate and analyse fuzzy logic and PID systems. The student is guided through four stages. The procedures, methodologies, results, analysis, discussion and conclusion utilised by the student is summarised in this report.

2. Problem Definition

2.1. Stage 1: Control System Simulation

In stage one, Simulink is used to employ a Proportional-Integral (PI) controller to control the velocity of an electric motor with the following transfer function used to represent the motor dynamics:

$$G(s) = \frac{500}{s(s^2 + 550s + 25000)}$$

Equation 1

where the input is u , and the output is y .

The motor load is described by the function:

$$f_{(r)}t = 9y^3(t)$$

Equation 2

The objective is to build the controller and fine tune the PI gains to achieve the **shortest rise time** (time taken for the system to go from 10 – 90% of steady state value) while attempting to keep the **maximum overshoot** (percentage by which the maximum value of the signal exceeds the steady-state value) **below 10%**. This is tested for the two scenarios - with and without the load.

2.2. Stage 2: Fuzzy Inference System

Fuzzy logic toolbox is introduced in stage two. The student is asked to follow the tutorial example provided in the *Build Fuzzy Systems Using Fuzzy Logic Designer* chapter to build a Fuzzy Inference System for a tipping problem using MATLAB. The student would then modify the parameters like the membership functions, interpretation operations and aggregation methods and discuss their observations.

2.3. Stage 3: Fuzzy Control of an Electric Motor

By applying the practice and knowledge from stage 2 about building a fuzzy logic controller, the student will approach the same task we had in stage 1, with a different solution. Stage 3 involves using a fuzzy logic controller instead of the PI controller for this non-linear system.

Like stage 1, the student will observe the velocity control response of the electric motor in two separate scenarios: i) with load, ii) without load. Each scenario is fine-tuned by changing the scaling parameters H0 (error scaling factor), H1 (derivative of error scaling factor) and H2 (controller output scaling).

The task requires a fuzzy system that has the shortest rise time with an overshoot less than 10%. Additionally, the controller must be designed with a reasonable rule base and ‘at least 5 linguistic values’ for each of the 3 linguistic variables. The student has a lot of flexibility on different characteristics of the system such as deciding the fuzzy control type and appropriate AND/OR, implication, aggregation and defuzzification methods, the membership function types and their range, and the input signal.

2.4. Stage 4: TSK Fuzzy Control of an Electric Motor

In the final stage, the student builds a system aiming to achieve the same task done by the control system built in stage 3. The only difference being that a Takagi-Sugeno-Kang (TSK) controller is used to deal with the non-linearity of the motor.

Before designing the TSK controller, we need to obtain the crisp outputs we will be linking the inputs to. PID systems are developed to determine the crisp outputs before interpolating them to the controller. Upon tuning and testing different parameters to meet the criteria of shortest rise time with an overshoot of less than 10%, each of the models are compared to one another to make note of their individual strengths and limitations.

3. Procedures & Methodologies

3.1. Stage 1: Control System Simulation

A step-input was used for simplicity (instead of a signal generator and constant) as we only needed to compare the rise-time and maximum overshoot. The PI controller being tested was connected next, along with the motor dynamics transfer function, an integrator to convert to velocity, and a scope to view the signal. The control system is built as seen by the *Figure 1*:

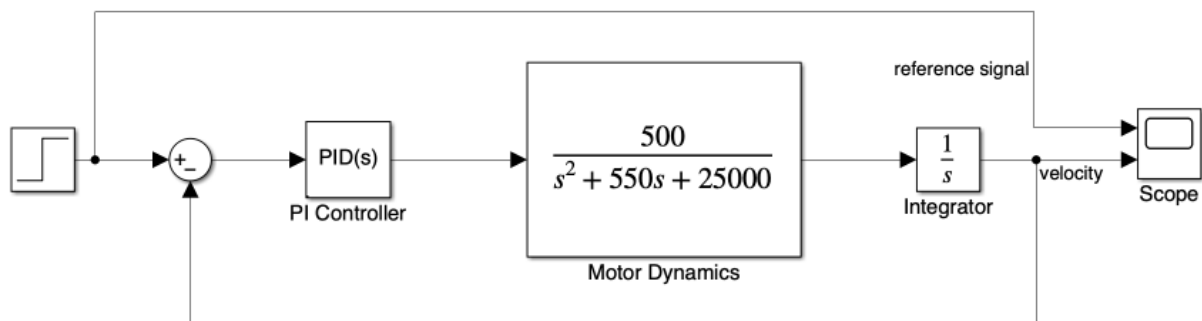


Figure 1 - PI system without load

Initially values were set as Proportional (P) = 10 and Integral (I) = 1. A spreadsheet was used to record the rise time and maximum overshoot observed to determine which values provided the best results.

The same approach was used for the PI system that includes the load. The load is added to the system to be subtracted from the input signal. The PI system with the load is displayed in *Figure 2*:

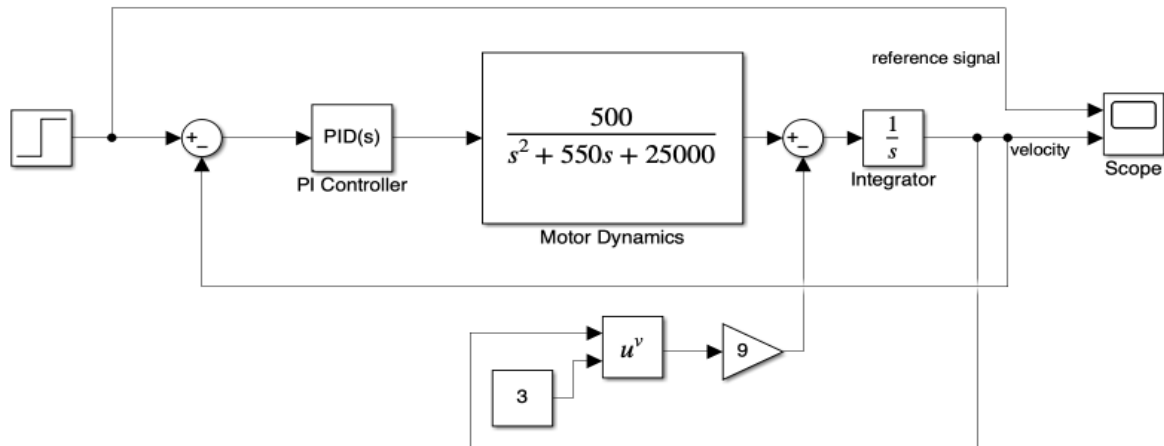


Figure 2 - PI system with load

3.2. Stage 2: Fuzzy Inference System

The MATLAB tutorial uses a template to create a type-1 Mamdani system, with min, max operators and a ‘centroid’ defuzzification method as seen by *Figure 13, Appendix B*.

We add two inputs – service and food, and a single output – tip. Each of them uses a different function shape from Gaussian, Trapezoidal and Triangular and parameters to simulate how they operate. These shapes can be changed later based on the intuition and experience of the student. For simplicity, only three rules are added, with the OR operator to incorporate the contribution of the ‘food’ input. The membership functions designed, and the rule base system can be seen from *Figures 14, 15, 16 & 17, Appendix B*.

Upon completion of the tutorial, the student is to make modifications to improve the tipper model to better meet the student’s expectations. The results are then compared to its previous iterations and workshop model. The student is introduced to the interface seen in *Figure 18, Appendix B*.

This interface lets us graphically observe how the output would react to the different inputs sent into the system. Similarly, a control surface may also be used to better visualise the overall shape of the input-output relationship. The control surface for the Mamdani FIS produced during the tutorial is displayed in *Figure 19, Appendix B*.

The tutorial showed how easy it is to convert a system to another using the Fuzzy Logic Designer, the Mamdani FIS was converted to a TSK (Sugeno) FIS and its control surface is shown in *Figure 20, Appendix B*.

3.3. Stage 3: Fuzzy Control of an Electric Motor

As per the advice from the assessment, the fuzzy control system is built by modifying the *sltankrule* system from MATLAB. This example from MATLAB is to control the fluid level of a liquid tank by using fuzzy logic. The tank is replaced with the motor, and controller is rebuilt to adjust the inputs 'Error' and 'Derivate of Error' to produce and output 'Control Signal' to have the motor best replicate the input reference signal.

The controller is built from a Mamdani Type-1 fuzzy inference system. The system was kept simple initially – the properties of the controller build are seen in *Figure 22, Appendix C*.

Figures 23 & 24, Appendix C - highlight how triangular membership types were used initially with a range of -1 to 1. These are to be edited later during the fine-tuning process for better representation. Each of the 3 linguist variables were given 5 different linguistic values – negative big, negative small, zero, positive small and positive big. Each value describes how the signal is acting (for inputs) or how the signal is meant to react (output). Based off these linguistic values, a rule base system was intuitively developed to match the inputs to an output. The rule base system had 25 different rules as displayed on *Figure 25, Appendix C*.

Finally, the fuzzy logic controller was integrated into a fuzzy logic system to send a signal and observe how the motor responds using fuzzy logic. The system built by the student is displayed in *Figure 3*:

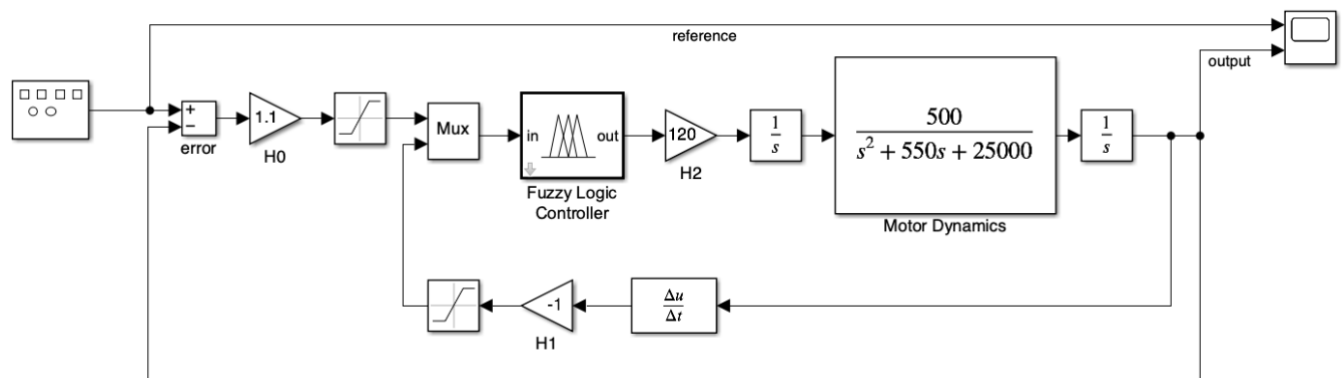


Figure 3 - Fuzzy logic control system for motor without load

- A signal generator inputs a signal with an amplitude of 1, which then subtracts the error signal observed from the output and is fed into the first tuning parameter H0.
- 'Graph blocks' are used to limit the input signal, so it always lies within the range of the membership function within the logic controller.
- This Error signal is fed into the fuzzy logic controller along with the Derivative of Error which is also initially passed through the H1 tuning parameter.
- The derivative of error (de/dt) is obtained from its equivalent output signal ($-dy/dt$). This relationship ($de/dt = -dy/dt$) assumes that the reference signal is a constant.
- The output obtained from the controller is scaled using the H2 parameter, which is then integrated and sent into the simulated motor.

- Finally, an integrator is used to obtain the velocity of the motor which is connected to a scope to compare the response of the motor to the reference signal that was initially inputted.
- As the system is closed loop, the feedback is returned to the signal generator input to adjust the new signal and the process is repeated.

The process is kept the same for scenario 2, however the load of the motor is also added to the simulation as seen on Figure 4:

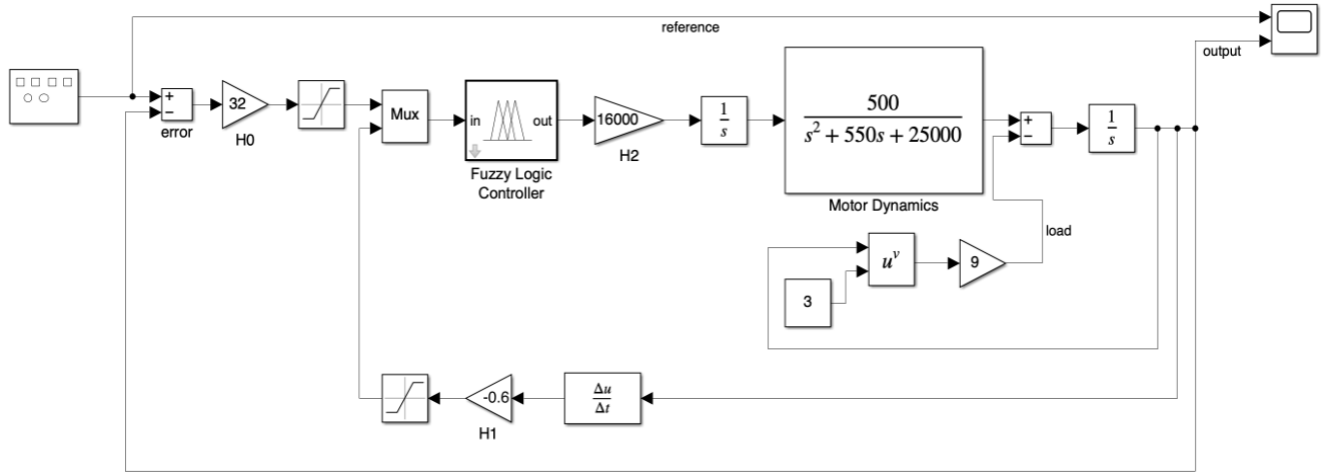


Figure 4 - Fuzzy logic control system for motor with load

3.4. Stage 4: TSK Fuzzy Control of an Electric Motor

As the criteria states that the maximum overshoot needs to be less than 10%, we can determine the range of output to be between -1.1 and 1.1. Therefore, the range is divided in 5 sections as denoted by Table 1:

Table 1 - Dividing the output to 5 sections

Section	Output Range	Midpoint
1	-1.1 to -0.66	-0.88
2	-0.66 to -0.22	-0.44
3	-0.22 to 0.22	0
4	0.22 to 0.66	0.44
5	0.66 to 1.1	0.88

PID controllers, like the one built in stage 1, scenario II is designed to input the mid-point values of each section as seen by Figure 28, Appendix D.

These controllers are then fine-tuned to output the shortest rise time while meeting the less than 10% overshoot criteria. Using the PI values for each of the sections, we use the following equation to determine their crisp output values:

$$output = midpoint \times (P + I)$$

The results are displayed in Table 2:

Table 2 - Tuning PI controllers for each midpoint

Midpoint	Proportional (P)	Integral (I)	Rise time	Max under/overshoot	Crisp Output
-0.88	1,700	12,550	39.3 ms	9.34%	-12540
-0.44	1,550	3,000	45.8 ms	9.34%	-2002
0	NA (1700)	NA (17,000)	NA (38 ms)	NA (9.34%)	0
0.44	1,550	3,000	45.8 ms	9.34%	2002
0.88	1,700	12,550	39.3 ms	9.34%	12540

Like the Mamdani controller, to keep the controller simple, we use 2 inputs with 5 triangular membership functions with a range of -1 to 1. The rule base system which is in conjunction with the output produces crisp results (constants determined by a function) instead of linguistic variables. The Figures 29, 30, 31 & 32, Appendix D display the properties defined for the TSK controller.

The TSK control system built by the student is seen in Figure 5:

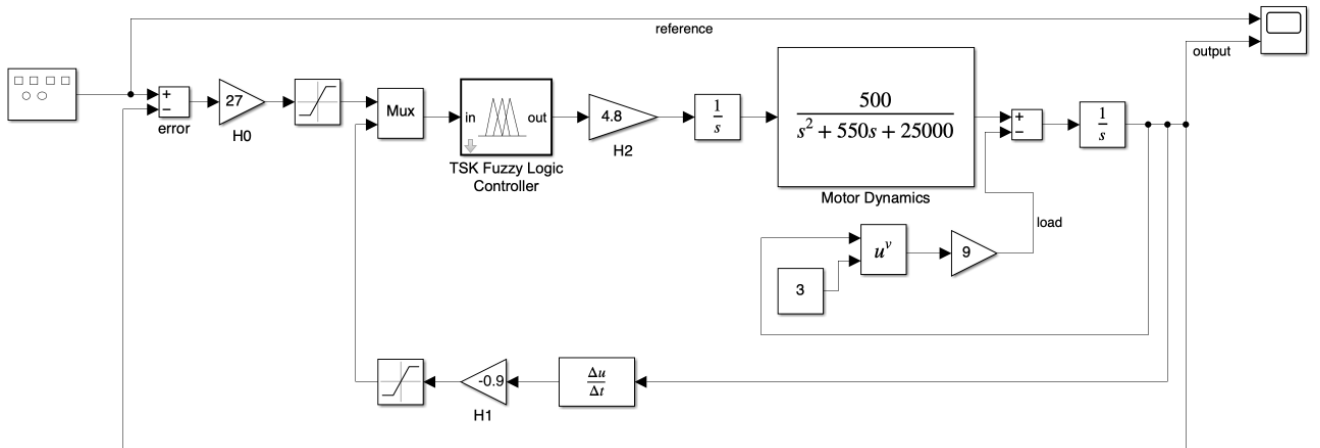


Figure 5 - TSK logic control system for motor with load

4. Results & Discussion

4.1. Stage 1: Control System Simulation

Comparing the two systems, with and without load – it was clear that the system with the load responded significantly slower. The simulation time had to be increased to get a proper measurement. Both systems were able to be significantly optimised.

Without load

Initially the system produced a rise time of 5.97 seconds with an overshoot of 21% which was over the criteria. It was found that increasing the P parameter would decrease the rise time but increase the overshoot. Whereas decreasing the I parameter would increase the rise time

slightly, however it would decrease the overshoot. The Table 3 shows some of the different values tested and the results obtained.

Table 3 - Results analysis, without load

Stage 1: without load				
P	I	Rise Time (s)	Overshoot (%)	Description
10	1	5.970	21.24	Initial results
1000	1000	0.069	6.99	Range found, used as a control
1500	1000	0.047	11.80	Increase P -> rt decreased, ovrshoot increased
500	1000	0.124	14.30	Decrease P -> rt increased, ovrshoot increased
1000	1500	0.067	9.34	Increase I -> rt dec (slight), overshoot inc
1000	500	0.071	4.74	Decrease I -> rt inc (slight), overshoot dec
1559	6	0.047	9.34	Good result
1550	53	0.047	9.34	Different options but results maxed out

It was observed that while different approaches could be taken to obtain the same result, the balance between the P and I parameters would eventually produce the same results. In this case, the student had chosen **P = 1550** and **I = 53**, to obtain a **rise time of 0.047 seconds** with an **overshoot of 9.34%**. The *Figure 11, Appendix A* display results obtained, and Figure 6 displays the scope view:

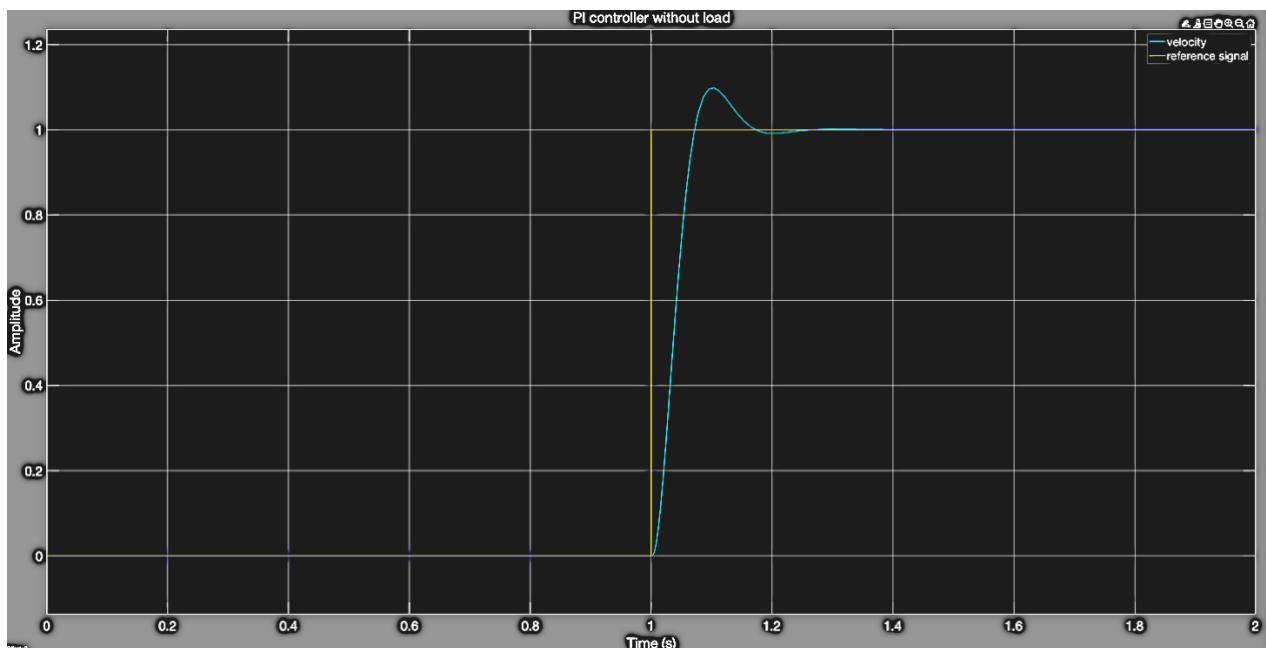


Figure 6 - Fine-tuned PI controller without load

With load

Initially the system responded extremely slow, with a rise time of 1330 seconds and minimal overshoot. The P and I parameters responded similarly to the system without the load however it was observed that if the I parameter isn't very high (around 10000), the system produced an

unstable response. It had oscillations before reaching the steady state value. Therefore, the I parameter had to be significantly increased.

Unlike the ‘without load’ system, it was also observed that if the P parameter is too high or if the I parameter is dropped too low, we also observe reverse oscillations (an undershoot after the overshoot). This also needed to be avoided to build a stable system and meet the criteria.

Table 4 - Results analysis, with load

Stage 1: with load				
P	I	Rise Time (s)	Overshoot (%)	Description
10	1	1330	0.26	Initial results
1000	1000	-	-	Unstable - overshoot before reaching steady state value
1000	10000	0.058	2.58	Range found, used as control
1500	10000	0.045	2.58	Increase P -> rt decreased, ovrshoot unaffected, reverse osci
500	10000	0.074	8.15	Decrease P -> rt increased, ovrshoot increased
1000	15000	0.051	9.34	Increase I -> rt dec (slight), overshoot inc
1000	9500	0.059	1.53	Decrease I -> rt inc (slight), overshoot dec, reverse osci
1700	17000	0.038	9.34	Good result, no unstability or reverse oscillations

The optimum solution tuned by the student used $P = 1,700$ and $I = 17,000$ resulting in a rise time of **0.038 seconds** and an **overshoot of 9.34%**. Very similar to the system without the load. The *Figure 12, Appendix A* displays the results and *Figure 7* displays the scope view:

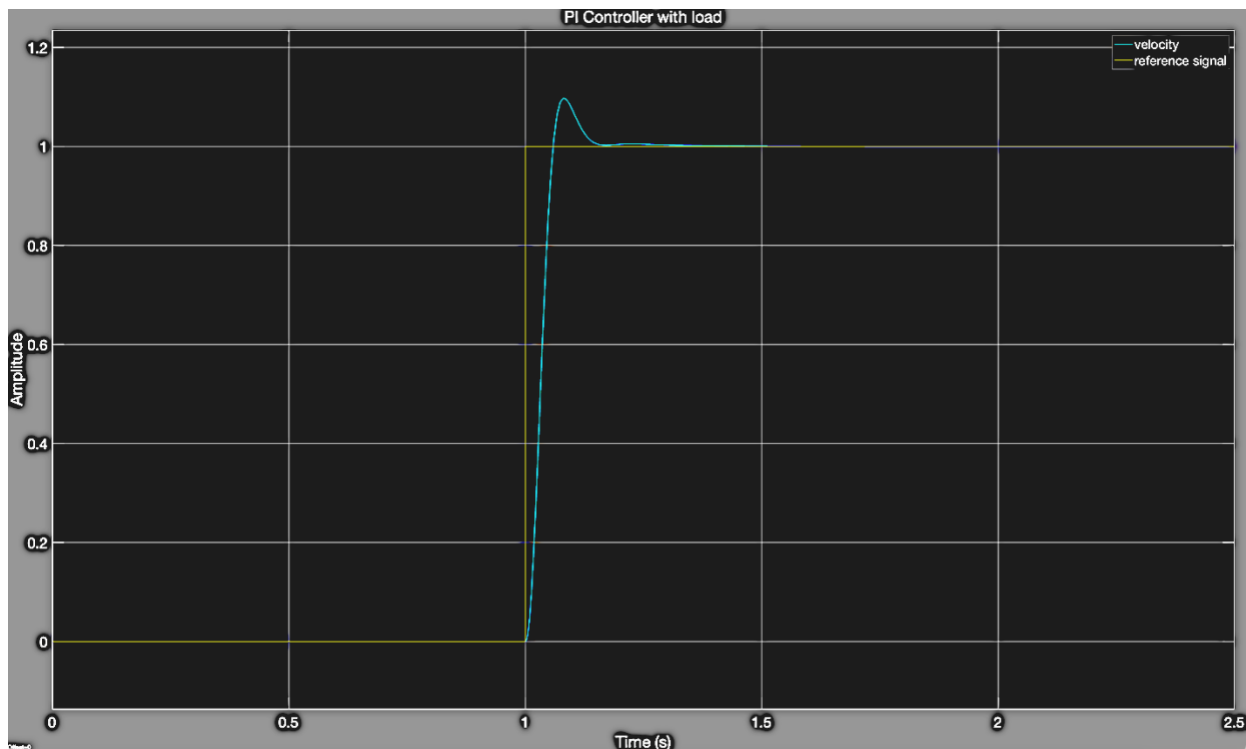


Figure 7 – Fine-tuned PI controller with load

4.2. Stage 2: Fuzzy Inference System

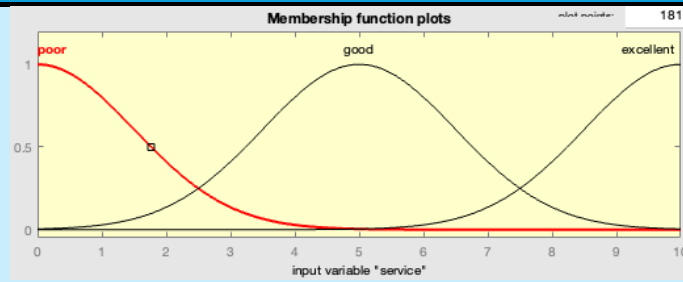
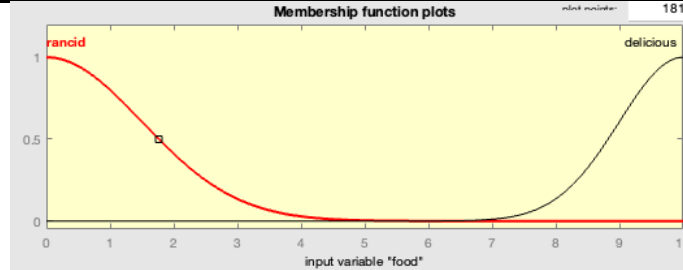
The tipper designed in the project was compared to the one created in workshop I. The membership functions and all the properties of both the systems were the same. However, the project tipper used 3 rules where an OR operator was used in a more effective way to incorporate the ‘food’ input within the ‘service’ inputs. The workshop I model used 5 rules, but the results observed were similar as both systems were asking the models to do the same thing in different formats. This is confirmed by the views provided in *Figure 21, Appendix B* for each of the systems:

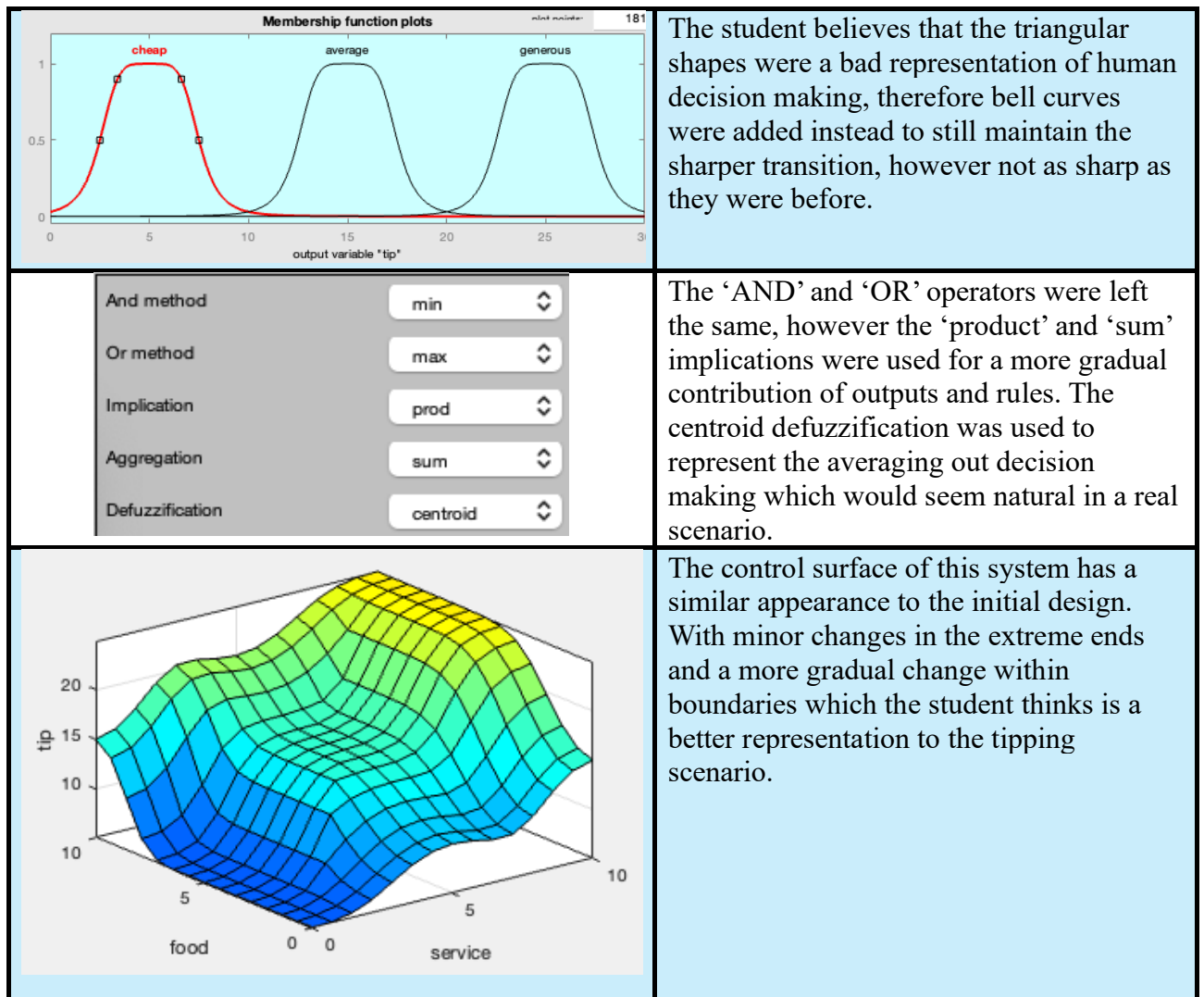
During the modification process, the student noted the following changes when changing the properties of the system and observing the control surface after each iteration:

- Triangular and trapezoidal membership functions added sharper transitions to the control surface whereas Gaussian or Bell-shaped made it more gradual and softer.
- Similarly, ‘min’ implication added more flatter, sharper changes whereas ‘product’ did the contrary.
- Aggregation methods such as ‘max’ caused abrupt jumps between some rule regions (borders), however ‘sum’ smoothened the surface. Except for extreme cases where it appeared to overshoot.
- In defuzzification, ‘centroid’ method showed the most stable outputs. The ‘Mean of Maximum’ method was a little more flatter and sharper between peaks.

Upon making refinements, the final model configuration is provided in Table 5:

Table 5 - Final model of tipping system using Mamdani FIS

Property	Description
	Membership function for ‘service’ for left the same as it represented well.
	Membership function for ‘food’ was changed to ‘Gaussian’ from ‘Trapezoidal’ to smoothen out the steep increase/decrease sections and eliminate the flat regions at the ends, as the student wants to represent a continuous influence on tip based even on the extreme ends.



4.3. Stage 3: Fuzzy Control of an Electric Motor

Tuning the Mamdani controller took a considerable amount of time, initially the system responded extremely slowly. However, when the output gain was increased, the system was extremely sensitive and produced a huge overshoot. The membership functions and the rule base system needed to be modified for its sensitivity, however a decent result was finally obtained with the following parameters for the controller without the load: **H0 = 1.1, H1 = 1, H2 = 120**.

During the tuning process however, the student was able to get a good understanding on how each parameter affect the system. It was observed that H0 (error scaling) made the system more aggressive, pushing the system to correct its error faster. While this helped shorten the rise time, it would in most cases lead to an overshoot. H1 (rate of error scaling) primarily helped minimise the oscillations after the maximum overshoot. While this didn't affect the criteria of the assessment, this was balanced to make the response match the reference signal in latter stages of the transient response. H2 (output scaling) behaved like an amplifier, it would desensitise the whole system, making the system unstable if given a value too high. This parameter had the biggest influence on the response and so was the hardest to tune.

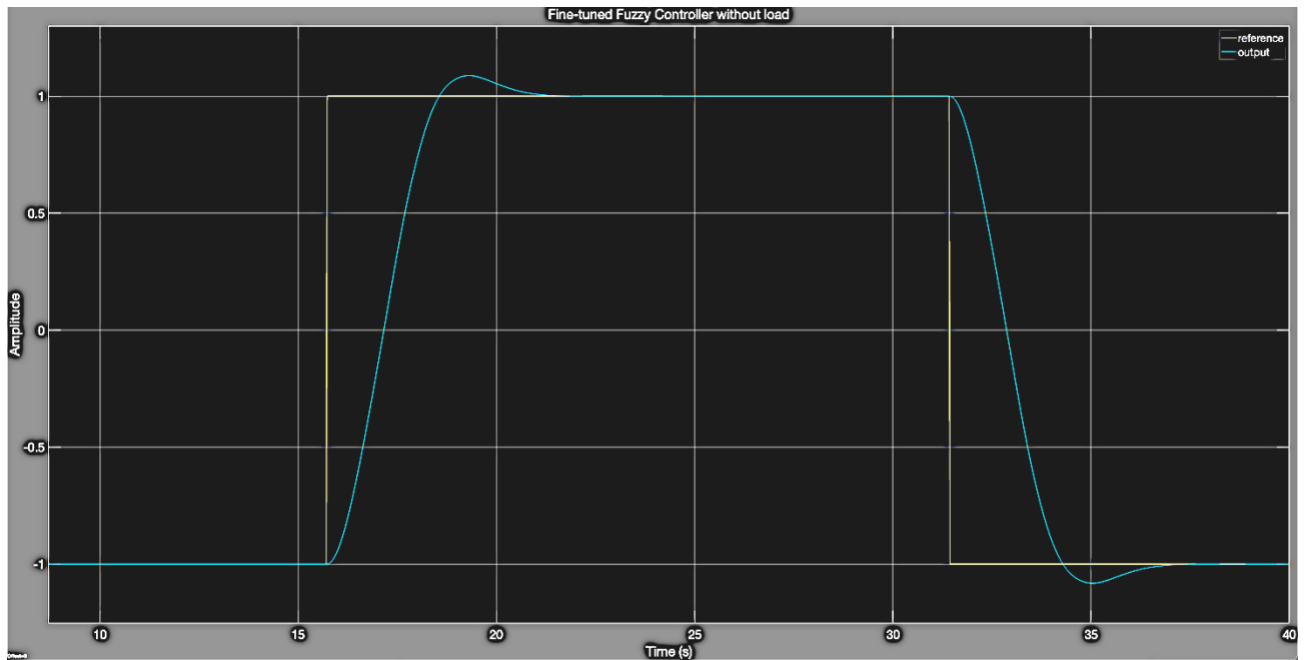


Figure 8 - Fine-tuned fuzzy control system without load

The student was eventually able to obtain a response that meets the criteria – with a slightly slower **rise time of 1.848 seconds** and a **maximum overshoot of 8.8%** for the motor controller without a load as seen on Figure 8.

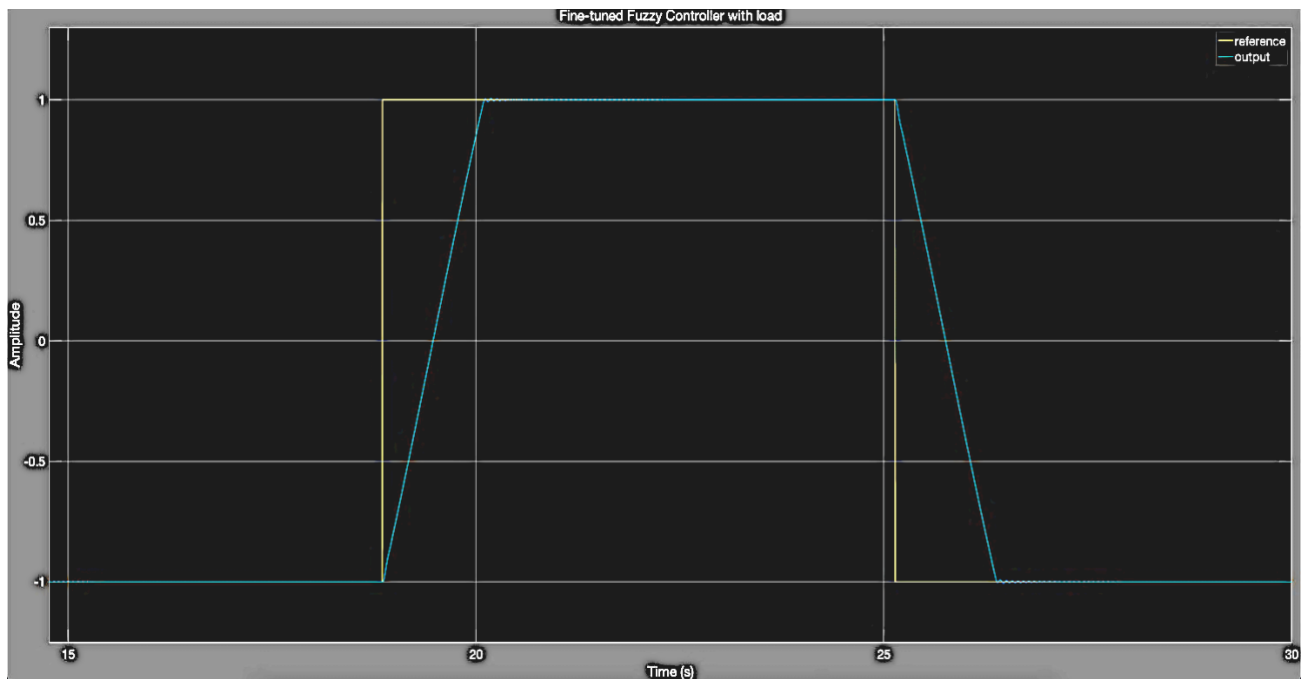


Figure 9 - Fine-tuned fuzzy control system with load

With the addition of the load, the significance of the overshoot was decreased in the response. Therefore, the signal was slightly easier to be tune by scaling up the output as a whole (using H2), the following parameters were used upon tuning: **H0 = 32, H1 = 0.6, H2 = 16,000**. This resulted in a rise time of **0.977 seconds** and a **maximum overshoot of 0.505%** - producing a sharp response as seen on Figure 9.

Additional testing was done by increasing the number of membership functions. This helped reduced the sharper response for the fuzzy system with the load. It was also negated the small oscillations/ noisy response at steady state. It was also noted that adding more membership functions to the system made it more straining for the computer to run the system.

The student was not able to achieve a response that was equivalent to the system that was observed initially. While this may be possible based off the flexibility observed by the Mamdani FIS, it would be extremely difficult. A closest result was obtained by using trapezoidal membership functions with a shorter range to mimic the triangular functions.

With the experience of the student, the Mamdani controller produced a response almost 1.8 seconds slower for the system without the load and about a second slower for the system with the load. While these results may not seem far off, it would be very time consuming and difficult to further bring the range closer to that of the PID. However, it can be further optimised. The Mamdani system demonstrated to be less affected by the introduction of a load in contrast to the PID controller which needed to be significantly retuned. This highlights the better robust nature of the Mamdani controller to non-linear disturbances.

Using alternative fuzzy inference systems slightly changed the output, each system had to be tuned using a trial-and-error approach as some controllers were unpredictable especially with different defuzzification methods.

4.4. Stage 4: TSK Fuzzy Control of an Electric Motor

The student was able to tune the response such that a **1.37 second rise time** and an extremely low **0.505% maximum overshoot** was obtained. While it was clear that the rise time could be further decreased with a higher overshoot, the student had difficulty damping the oscillations created that wouldn't stabilised. The scope view of the response is displayed in Figure 10:

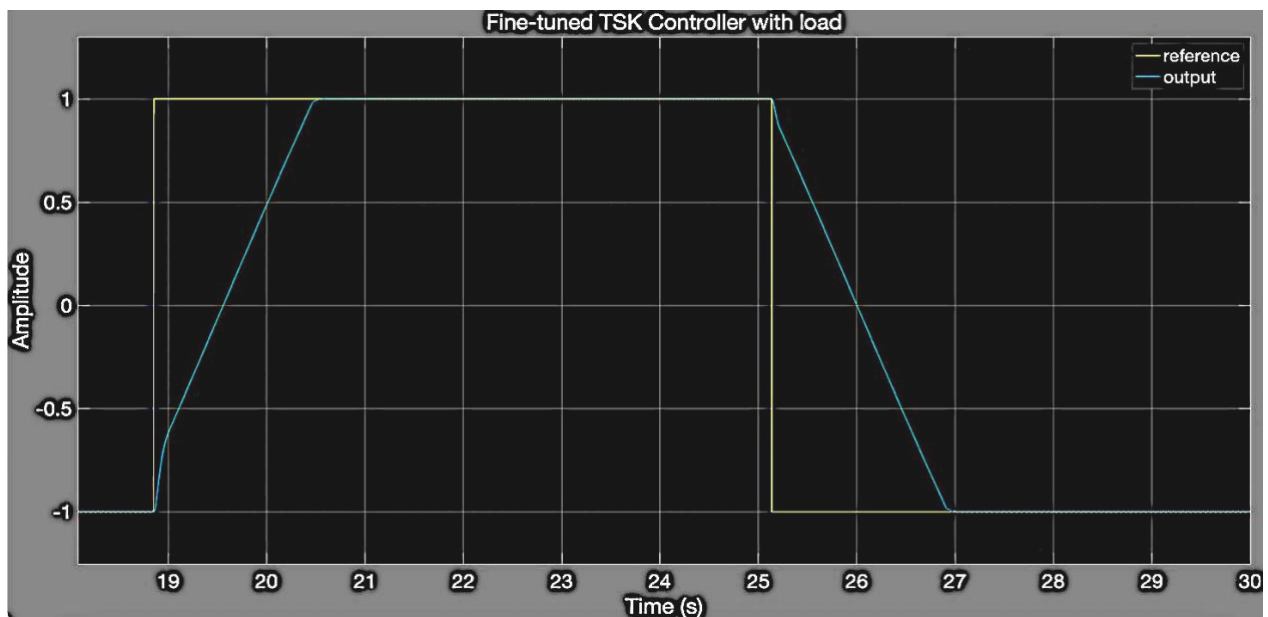


Figure 10 – Fine-tuned TSK control system with load

The same system was tested for different membership function and alternating the rules, as expected, smoother functions like Gaussian or Bell made the interpolation smoother in comparison to the sharp results from the Triangular functions as seen from the control surface - *Figure 33, Appendix D*. The rules didn't seem to make much of a difference on the output, likely because the output gain was H2 was much lower (4.8) for this controller.

It is likely that with more experience the TSK controller could be adjusted to offer a faster rise time in comparison to a single global PI controller in non-linear regions. However, TSK controller depend heavily on accurate linear approximations. Additionally, it is much harder to implement in comparison to a direct PID controller.

Although Mamdani produced a better rise time, the TSK controller seemed a little easier to tune than the Mamdani controller, as the outputs were crisp outputs and gave us a better sense on how the output may affect the motor. The output also produced a more adaptive response across the operation range, unlike the Mamdani which was very sensitive especially with its overshoot.

5. Conclusion

The assessment provided a student a good practical understanding on the PID, Mamdani and TSK controller. From building each the controllers, the strengths and limitations were addressed and compared with one another. It was clear that PID controller were the easiest to implement and tune, and therefore in this assessment, they produced the best results with a rise time of **0.047 seconds** for with load and **0.038 seconds** for without load, each with an overshoot of 9.34%. However, it was argued that this is likely because both Mamdani and TSK controller have many variables that may produced better results if tuned properly. The flexibility provided by both the fuzzy controllers allow them to perform better especially in non-linear regions in return for the difficulty of implementation.

6. Appendix

A. Stage 1: Control System Simulation

+ Rise Time	47.211 ms
+ Slew Rate	16.941 (/s)
- Edges	0
- Fall Time	--
- Slew Rate	--
▼ Overshoots / Undershoots	
+ Preshoot	0.549 %
+ Overshoot	9.341 %

Figure 11 - Rise time and maximum overshoot of fine-tuned PI controller (without load)

+ Rise Time	38.121 ms
+ Slew Rate	20.968 (/s)
- Edges	0
- Fall Time	--
- Slew Rate	--
▼ Overshoots / Undershoots	
+ Preshoot	0.549 %
+ Overshoot	9.341 %

Figure 12 - Rise time and maximum overshoot of fine-tuned PI controller (with load)

B. Stage 2: Fuzzy Inference System

Type:	Mamdani Type-1
Name	tipper
And method	min ▼
Or method	max ▼
Implication method	min ▼
Aggregation method	max ▼
Defuzzification method	centroid ▼
Inputs:	2
Outputs:	1
Rules:	9

Figure 13 - Defining properties of Mamdani FIS - tipper

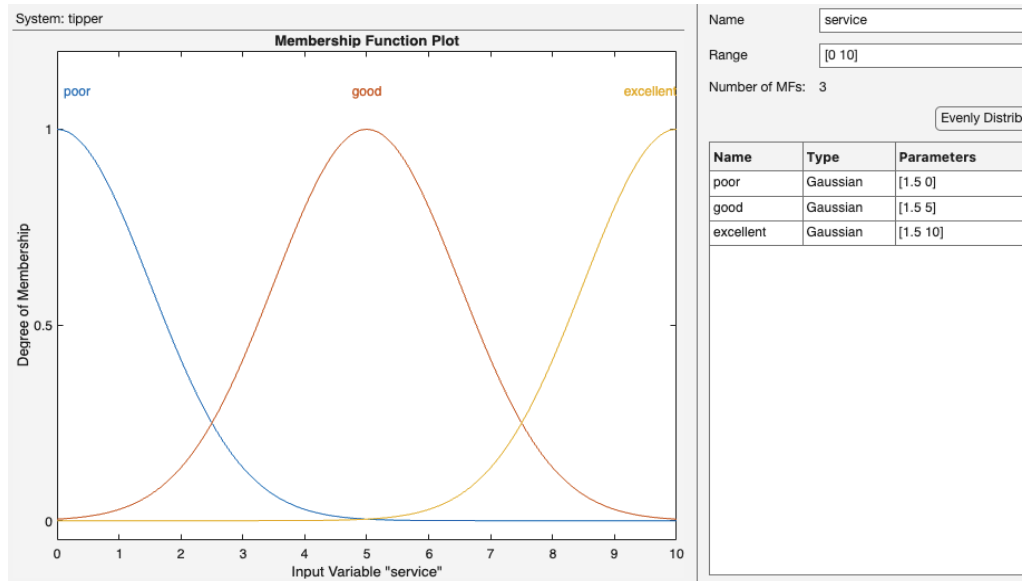


Figure 14 - Defining membership function for service (input 1)

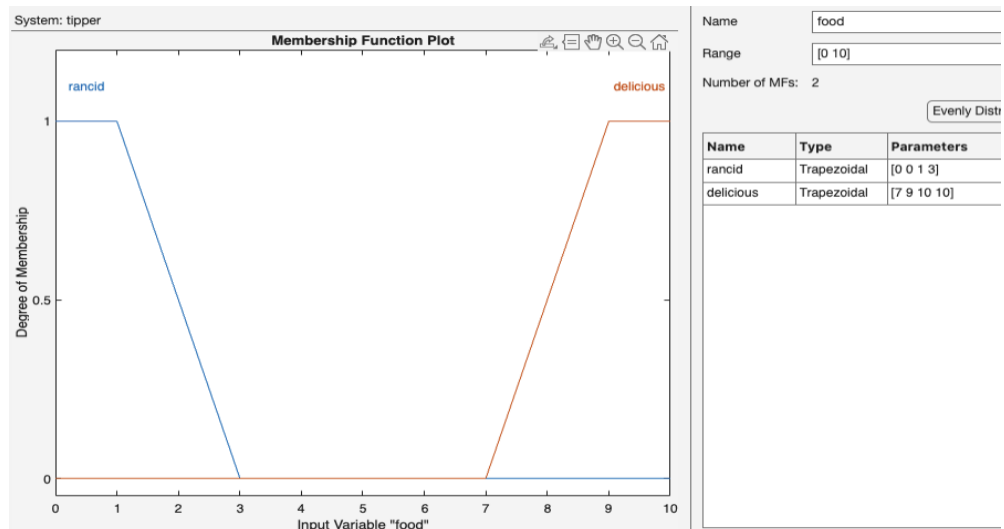


Figure 15 - Defining membership functions for food (input 2)

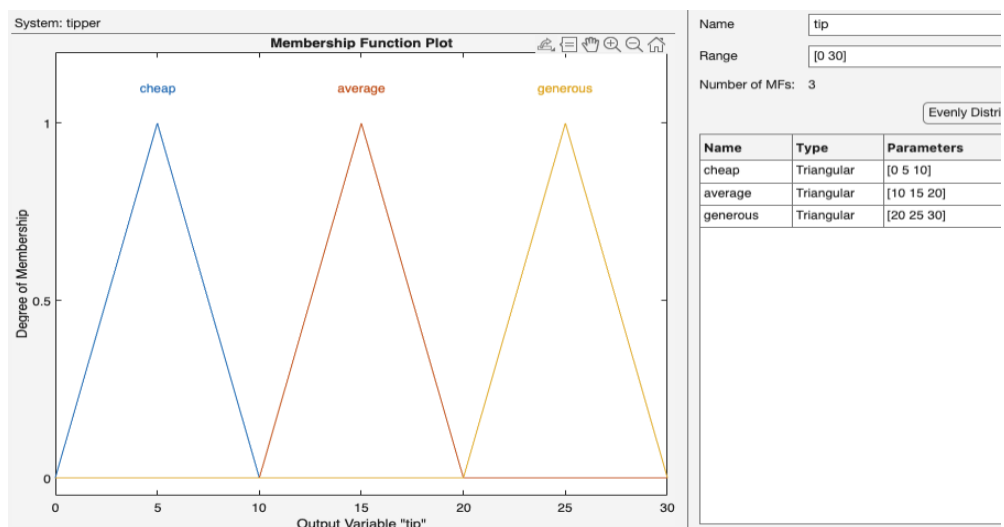


Figure 16 - Defining membership functions for tip (output)

	Rule
1	If service is poor or food is rancid then tip is cheap
2	If service is good then tip is average
3	If service is excellent or food is delicious then tip is generous

Figure 17 – Defining rule base for FIS

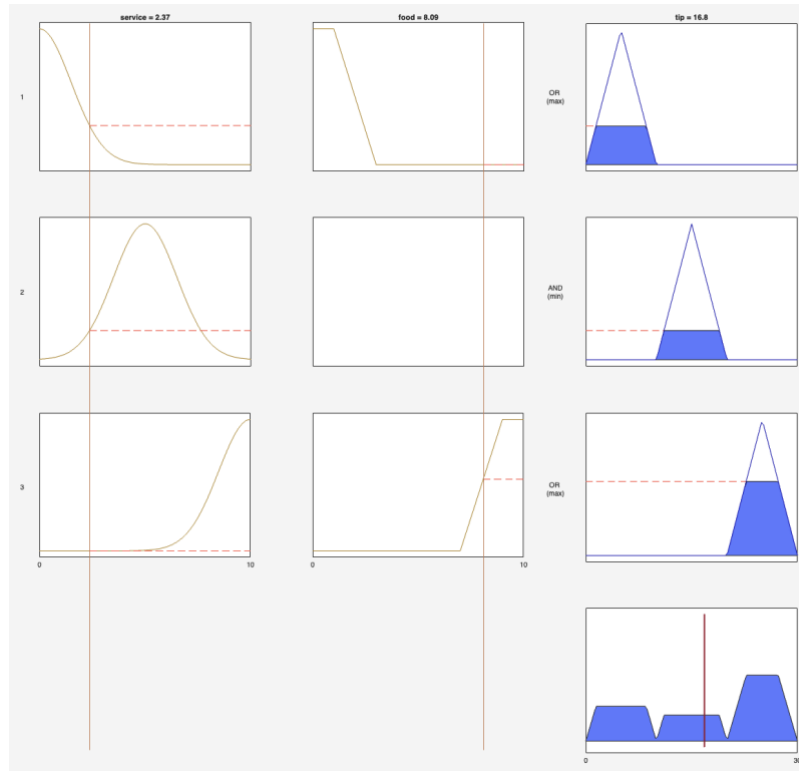


Figure 18 - Observing rule inference process for different inputs

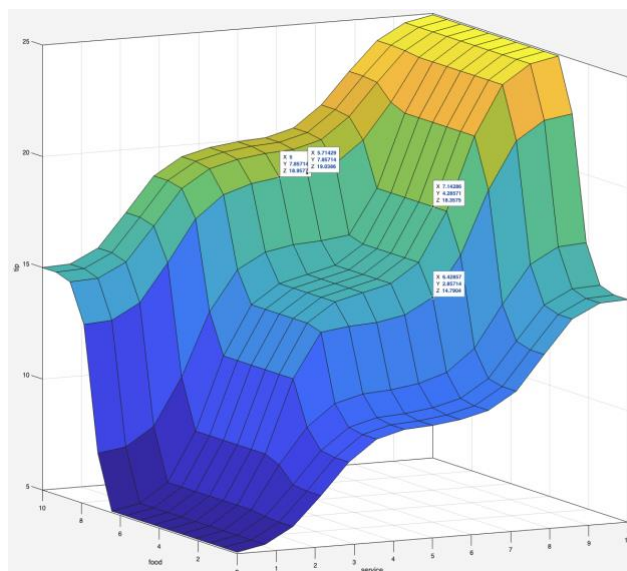


Figure 19 – Generating control surface for inputs vs output of the Mamdani FIS

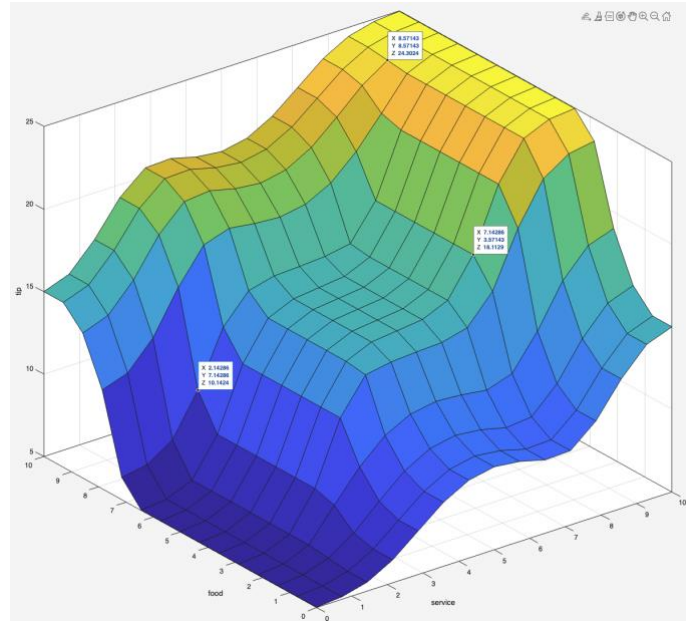


Figure 20 – Converting Mamdani control surface to a TSK (Sugeno) FIS

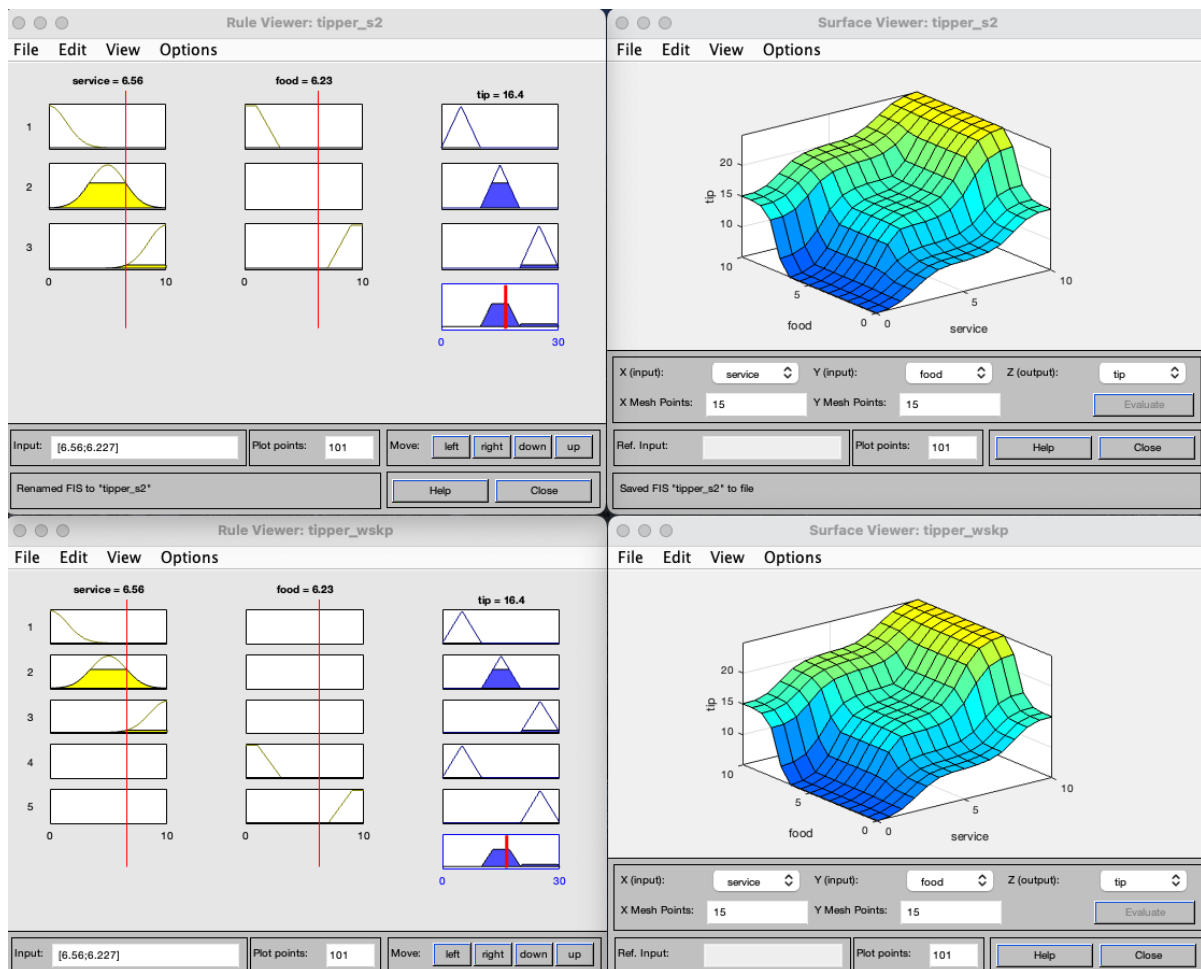


Figure 21 - Comparison between tipping system in project (top) and workshop I (bottom)

C. Stage 3: Fuzzy Control of an Electric Motor

Type:	Mamdani Type-1
Name	motor_control
And method	min
Or method	max
Implication method	min
Aggregation method	max
Defuzzification method	centroid
Inputs:	2
Outputs:	1
Rules:	25

Figure 22 - Defining properties of Mamdani FIS - motor_control

Name	Error	Name	Derivative of Error		
Range	[-1 1]	Range	[-1 1]		
Number of MFs: 5		Number of MFs: 5			
Name	Type	Parameters	Name	Type	Parameters
Negative Big	Triangular	[-1 -1 -0.5]	Negative Big	Triangular	[-1 -1 -0.5]
Negative Small	Triangular	[-1 -0.5 0]	Negative Small	Triangular	[-1 -0.5 0]
Zero	Triangular	[-0.5 0 0.5]	Zero	Triangular	[-0.5 0 0.5]
Positive Small	Triangular	[0 0.5 1]	Positive Small	Triangular	[0 0.5 1]
Positive Big	Triangular	[0.5 1 1]	Positive Big	Triangular	[0.5 1 1]

Figure 23 - Membership functions for the inputs

Name

Control Signal

Range

[-1 1]

Number of MFs: 5

Name	Type	Parameters
Negative Big	Triangular	[-1 -1 -0.5]
Negative Small	Triangular	[-1 -0.5 0]
Zero	Triangular	[-0.5 0 0.5]
Positive Small	Triangular	[0 0.5 1]
Positive Big	Triangular	[0.5 1 1]

Figure 24 - Membership functions for the output

	Rule
1	If Error is Negative Big and Derivative of Error is Negative Big then Control Signal is Negative Big
2	If Error is Negative Big and Derivative of Error is Negative Small then Control Signal is Negative Big
3	If Error is Negative Big and Derivative of Error is Zero then Control Signal is Negative Big
4	If Error is Negative Big and Derivative of Error is Positive Small then Control Signal is Negative Small
5	If Error is Negative Big and Derivative of Error is Positive Big then Control Signal is Zero
6	If Error is Negative Small and Derivative of Error is Negative Big then Control Signal is Negative Big
7	If Error is Negative Small and Derivative of Error is Negative Small then Control Signal is Negative Small
8	If Error is Negative Small and Derivative of Error is Zero then Control Signal is Negative Small
9	If Error is Negative Small and Derivative of Error is Positive Small then Control Signal is Zero
10	If Error is Negative Small and Derivative of Error is Positive Big then Control Signal is Positive Small
11	If Error is Negative Big and Derivative of Error is Zero then Control Signal is Negative Big
12	If Error is Zero and Derivative of Error is Negative Small then Control Signal is Negative Small
13	If Error is Zero and Derivative of Error is Zero then Control Signal is Zero
14	If Error is Zero and Derivative of Error is Positive Small then Control Signal is Positive Small
15	If Error is Zero and Derivative of Error is Positive Big then Control Signal is Positive Big
16	If Error is Positive Small and Derivative of Error is Negative Big then Control Signal is Negative Small
17	If Error is Positive Small and Derivative of Error is Negative Small then Control Signal is Zero
18	If Error is Positive Small and Derivative of Error is Zero then Control Signal is Positive Small
19	If Error is Positive Small and Derivative of Error is Positive Small then Control Signal is Positive Small
20	If Error is Positive Small and Derivative of Error is Positive Big then Control Signal is Positive Big
21	If Error is Positive Big and Derivative of Error is Negative Big then Control Signal is Zero
22	If Error is Positive Big and Derivative of Error is Negative Small then Control Signal is Positive Small
23	If Error is Positive Big and Derivative of Error is Zero then Control Signal is Positive Big
24	If Error is Positive Big and Derivative of Error is Positive Small then Control Signal is Positive Big
25	If Error is Positive Big and Derivative of Error is Positive Big then Control Signal is Positive Big

Figure 25 - Rule base system for the motor control FIS

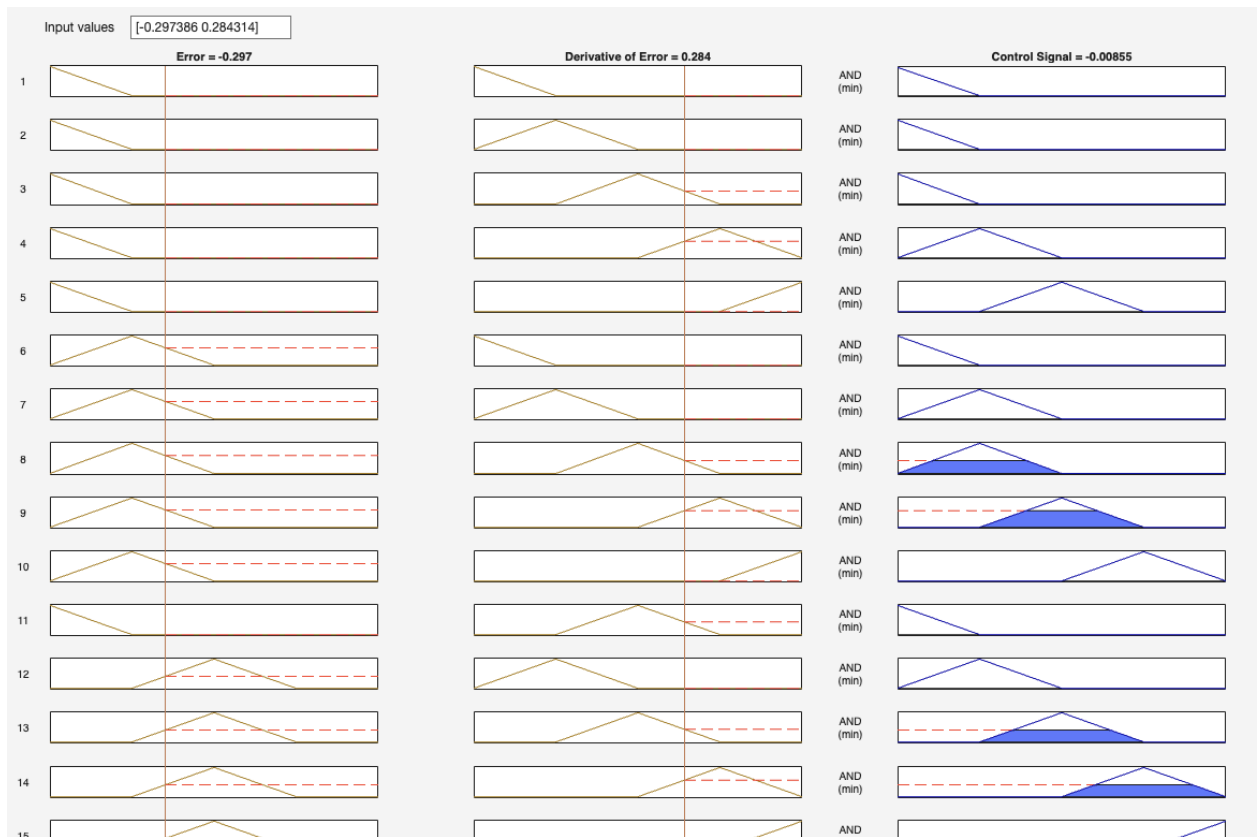


Figure 26 - Observing rule inference process for motor control

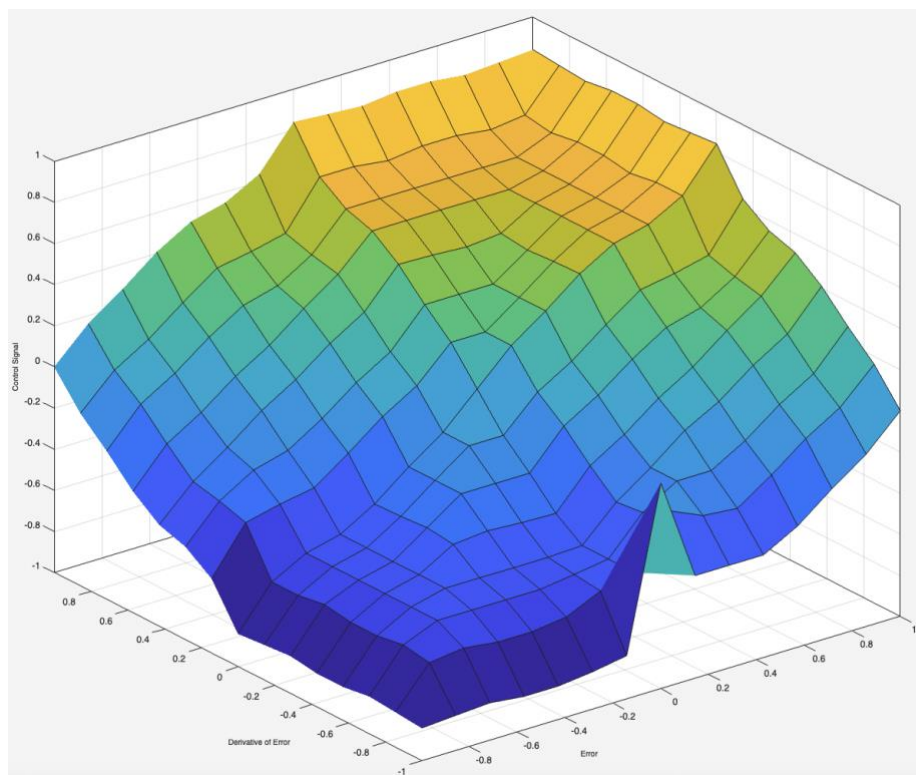


Figure 27 - Control surface of fuzzy logic controller for motor control

D. Stage 4: TSK Fuzzy Control of an Electric Motor

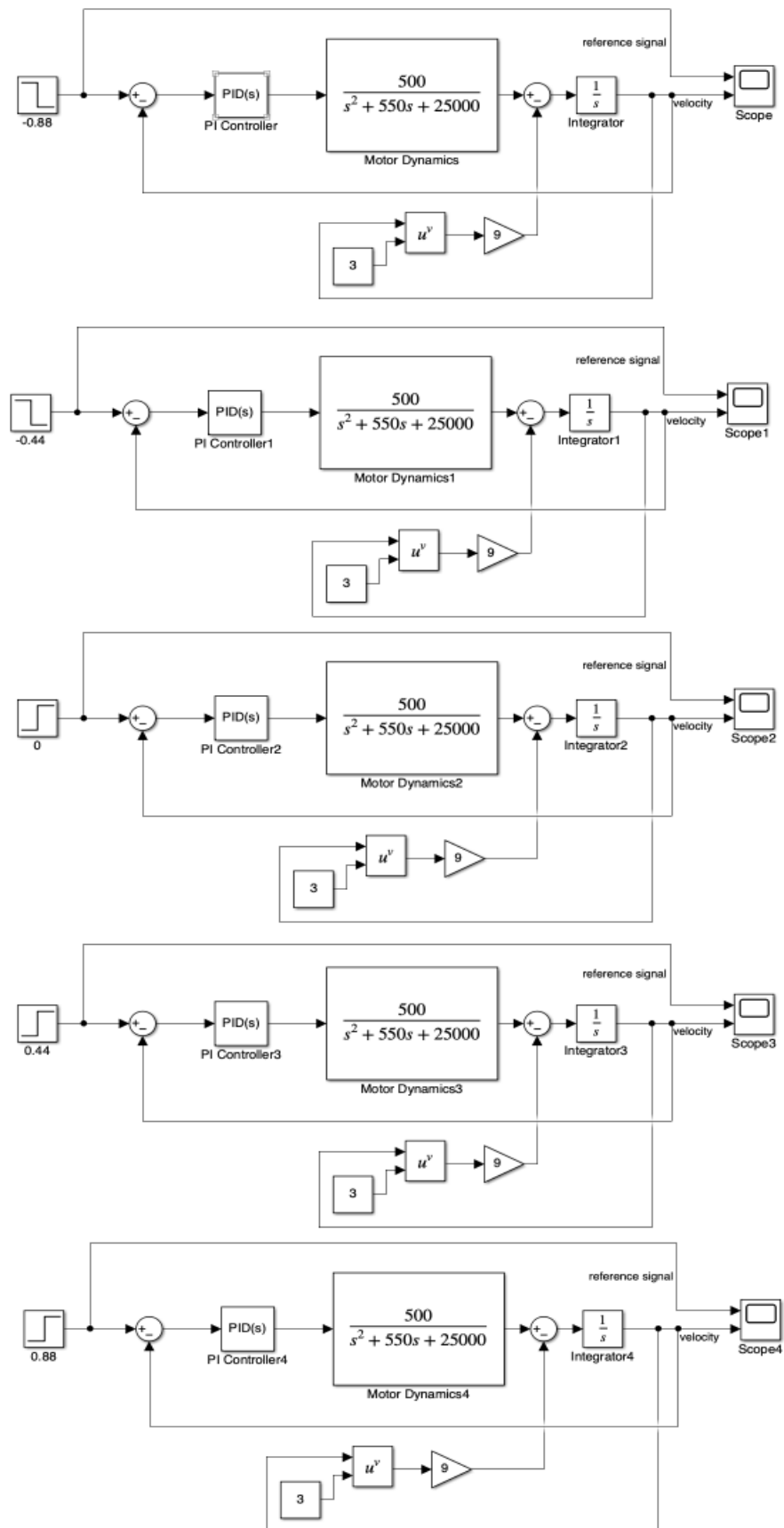


Figure 28 - 5 PI controllers for each section

Type: Sugeno Type-1

Name:

And method: ▼

Or method: ▼

Implication method: ▼

Aggregation method: ▼

Defuzzification method: ▼

Inputs: 2

Outputs: 1

Rules: 25

Figure 29 - Defining properties of TSK FIS - tsk_motor_control

Name

Error

Range

[-1.1 1.1]

Number of MFs: 5

Name	Type	Parameters
Negative Big	Triangular	[-1 -1 -0.5]
Negative Small	Triangular	[-1 -0.5 0]
Zero	Triangular	[-0.5 0 0.5]
Positive Small	Triangular	[0 0.5 1]
Positive Big	Triangular	[0.5 1 1]

Name

Derivative of Error

Range

[-1.1 1.1]

Number of MFs: 5

Name	Type	Parameters
Negative Big	Triangular	[-1 -1 -0.5]
Negative Small	Triangular	[-1 -0.5 0]
Zero	Triangular	[-0.5 0 0.5]
Positive Small	Triangular	[0 0.5 1]
Positive Big	Triangular	[0.5 1 1]

Figure 30 - Membership functions for the inputs

Name

Control Signal

Range

[-1.1 1.1]

Number of MFs: 5

Name	Type	Parameters
-12540	Constant ▼	-12540
-2002	Constant ▼	-2002
0	Constant ▼	0
2002	Constant ▼	2002
12540	Constant ▼	12540

Figure 31 - Output constants (crisp output)

	Rule
1	If Error is Negative Big and Derivative of Error is Negative Big then Control Signal is -12540
2	If Error is Negative Big and Derivative of Error is Negative Small then Control Signal is -12540
3	If Error is Negative Big and Derivative of Error is Zero then Control Signal is -12540
4	If Error is Negative Big and Derivative of Error is Positive Small then Control Signal is -2002
5	If Error is Negative Big and Derivative of Error is Positive Big then Control Signal is 0
6	If Error is Negative Small and Derivative of Error is Negative Big then Control Signal is -12540
7	If Error is Negative Small and Derivative of Error is Negative Small then Control Signal is -2002
8	If Error is Negative Small and Derivative of Error is Zero then Control Signal is -2002
9	If Error is Negative Small and Derivative of Error is Positive Small then Control Signal is 0
10	If Error is Negative Small and Derivative of Error is Positive Big then Control Signal is 2002
11	If Error is Negative Big and Derivative of Error is Zero then Control Signal is -12540
12	If Error is Zero and Derivative of Error is Negative Small then Control Signal is -2002
13	If Error is Zero and Derivative of Error is Zero then Control Signal is 0
14	If Error is Zero and Derivative of Error is Positive Small then Control Signal is 2002
15	If Error is Zero and Derivative of Error is Positive Big then Control Signal is 12540
16	If Error is Positive Small and Derivative of Error is Negative Big then Control Signal is -2002
17	If Error is Positive Small and Derivative of Error is Negative Small then Control Signal is 0
18	If Error is Positive Small and Derivative of Error is Zero then Control Signal is 2002
19	If Error is Positive Small and Derivative of Error is Positive Small then Control Signal is 2002
20	If Error is Positive Small and Derivative of Error is Positive Big then Control Signal is 12540
21	If Error is Positive Big and Derivative of Error is Negative Big then Control Signal is 0
22	If Error is Positive Big and Derivative of Error is Negative Small then Control Signal is 2002
23	If Error is Positive Big and Derivative of Error is Zero then Control Signal is 12540
24	If Error is Positive Big and Derivative of Error is Positive Small then Control Signal is 12540
25	If Error is Positive Big and Derivative of Error is Positive Big then Control Signal is 12540

Figure 32 - Rule base system for the TSK motor control FIS (crisp outputs)

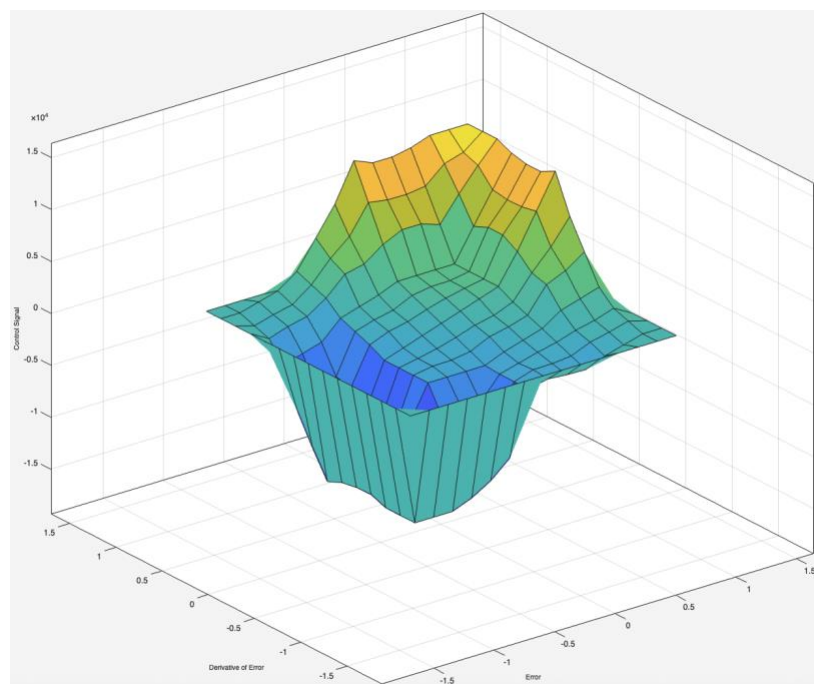


Figure 33 - Control surface of TSK controller for motor control